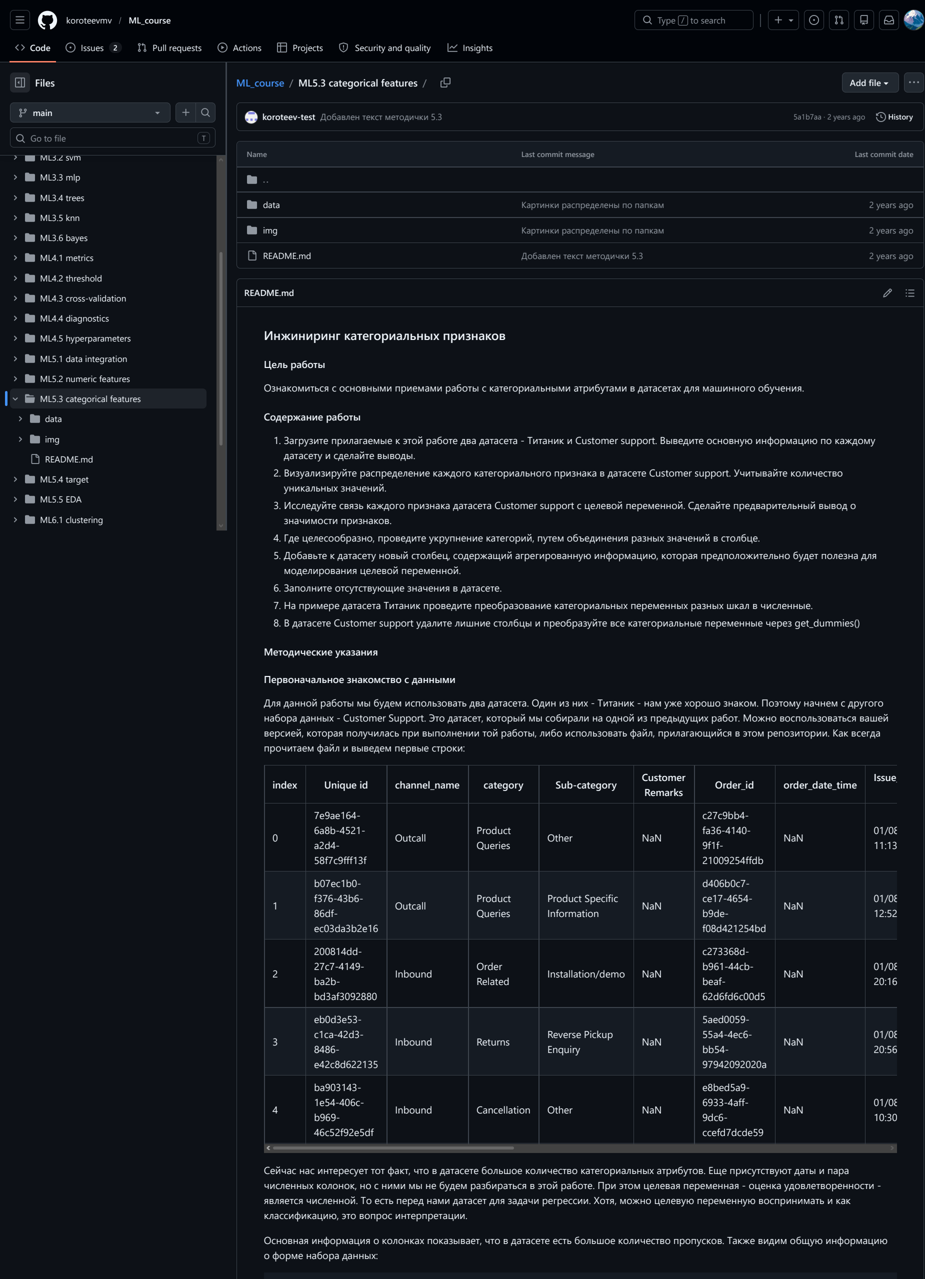


Files

main

Go to file T

ML3.2 svm
ML3.3 mlp
ML3.4 trees
ML3.5 knn
ML3.6 bayes
ML4.1 metrics
ML4.2 threshold
ML4.3 cross-validation
ML4.4 diagnostics
ML4.5 hyperparameters
ML5.1 data integration
ML5.2 numeric features
ML5.3 categorical features
  data
  img
  README.md
ML5.4 target
ML5.5 EDA
ML6.1 clustering

ML_course / ML5.3 categorical features /

Add file

koroteev-test Добавлен текст методички 5.3 5a1b7aa · 2 years ago History

Name	Last commit message	Last commit date
..		
data	Картинки распределены по папкам	2 years ago
img	Картинки распределены по папкам	2 years ago
README.md	Добавлен текст методички 5.3	2 years ago

README.md

Инжиниринг категориальных признаков

Цель работы

Ознакомиться с основными приемами работы с категориальными атрибутами в датасетах для машинного обучения.

Содержание работы

1. Загрузите прилагаемые к этой работе два датасета - Титаник и Customer support. Выведите основную информацию по каждому датасету и сделайте выводы.
2. Визуализируйте распределение каждого категориального признака в датасете Customer support. Учитывайте количество уникальных значений.
3. Исследуйте связь каждого признака датасета Customer support с целевой переменной. Сделайте предварительный вывод о значимости признаков.
4. Где целесообразно, проведите укрупнение категорий, путем объединения разных значений в столбце.
5. Добавьте к датасету новый столбец, содержащий агрегированную информацию, которая предположительно будет полезна для моделирования целевой переменной.
6. Заполните отсутствующие значения в датасете.
7. На примере датасета Титаник проведите преобразование категориальных переменных разных шкал в численные.
8. В датасете Customer support удалите лишние столбцы и преобразуйте все категориальные переменные через get_dummies()

Методические указания

Первоначальное знакомство с данными

Для данной работы мы будем использовать два датасета. Один из них - Титаник - нам уже хорошо знаком. Поэтому начнем с другого набора данных - Customer Support. Это датасет, который мы собирали на одной из предыдущих работ. Можно воспользоваться вашей версией, которая получилась при выполнении той работы, либо использовать файл, прилагающийся в этом репозитории. Как всегда прочитаем файл и выведем первые строки:

index	Unique id	channel_name	category	Sub-category	Customer Remarks	Order_id	order_date_time	Issue_
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Other	NaN	c27c9bb4-fa36-4140-9f1f-21009254ffdb	NaN	01/0 11:13
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	NaN	d406b0c7-ce17-4654-b9de-f08d421254bd	NaN	01/0 12:52
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/demo	NaN	c273368d-b961-44cb-beaf-62d6fd6c00d5	NaN	01/0 20:16
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	NaN	5aed0059-55a4-4ec6-bb54-97942092020a	NaN	01/0 20:56
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Other	NaN	e8bed5a9-6933-4aff-9dc6-ccefd7dcde59	NaN	01/0 10:30

Сейчас нас интересует тот факт, что в датасете большое количество категориальных атрибутов. Еще присутствуют даты и пара численных колонок, но с ними мы не будем разбираться в этой работе. При этом целевая переменная - оценка удовлетворенности - является численной. То есть перед нами датасет для задачи регрессии. Хотя, можно целевую переменную воспринимать и как классификацию, это вопрос интерпретации.

Основная информация о колонках показывает, что в датасете есть большое количество пропусков. Также видим общую информацию о форме набора данных:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 85907 entries, 0 to 85906
Data columns (total 20 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   Unique id                85907 non-null  object 
 1   channel_name             85907 non-null  object 
 2   category                 85907 non-null  object 
 3   Sub-category             85907 non-null  object 
 4   Customer Remarks         28742 non-null  object 
 5   Order_id                 67675 non-null  object 
 6   order_date_time          17214 non-null  object 
 7   Issue_reported at        85907 non-null  object 
 8   issue_responded          85907 non-null  object 
 9   Survey_response_Date     85907 non-null  object 
 10  Customer_City            17079 non-null  object 
 11  Product_category         17196 non-null  object 
 12  Item_price               17206 non-null  float64
 13  connected_handling_time  242 non-null    float64
 14  Agent_name               85907 non-null  object 
 15  Supervisor               85907 non-null  object 
 16  Manager                  85907 non-null  object 
 17  Tenure Bucket            85907 non-null  object 
 18  Agent Shift              85907 non-null  object 
 19  CSAT Score               85907 non-null  int64  
dtypes: float64(2), int64(1), object(17)
memory usage: 13.1+ MB
```

Как обычно, после визуального ознакомления с данными, нам нужно вывести статистическую информацию об атрибутах. Но так как по умолчанию функция describe описывает только численные столбцы, необходимо специально указать, что нас интересует информация по всем колонкам таблицы:

```
CS_data.describe(include="all")
```

Данная инструкция выводит уже знакомую нам статистическую информацию по численным колонкам, а по категориальным (то есть, технически - текстовым) выводит количество заполненных значений, количество уникальных значений, модальное значение и частоту этого самого вероятного значения:

index	Unique id	channel_name	category	Sub-category	Customer Remarks	Order_id	order_date_time	Issue_reported at
count	85907	85907	85907	85907	28742	67675	17214	85907
unique	85907	3	12	57	18231	67675	13766	30923
top	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Inbound	Returns	Reverse Pickup Enquiry	Good	c27c9bb4-fa36-4140-9f1f-21009254ffdb	09/08/2023 11:55	15/08/2023 10:59
freq	1	68142	44097	22389	1390	1	7	13
mean	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Самое полезное в этой таблице - соотношение количества уникальных к общему количеству значений по колонке. Если они совпадают или близки, это значит, что в атрибуте все или почти все значения разные. В таком случае стоит задуматься о его удалении, либо преобразовании. Также важно количество самого вероятного значения: этот показатель дает информацию о дисбалансе значений. Но более подробную информацию можно получить из анализа распределения атрибута.

Описательную статистику по датасету Титаник приведем еще в более компактном виде:

```
T_data.describe(exclude=[np.number])
```

Данная инструкция выводит описание только категориальных атрибутов датасета:

index	Name	Ticket	Cabin	Embarked
count	891	891	204	889
unique	891	681	147	3
top	Braund, Mr. Owen Harris	347082	B96 B98	S
freq	1	7	4	644

Здесь явно видно, например, что имя - полностью уникальный атрибут. А вот в одной каюте могло находиться до четырех человек. Что удивительно, номера билетов почему-то тоже повторяются.

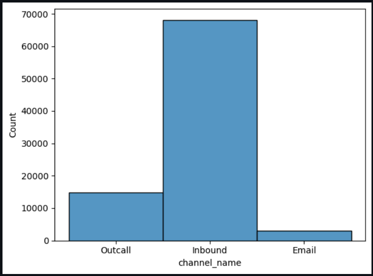
Обратите внимание но то, как в нашем варианте этого датасета обозначается класс обслуживания: названием, а не числом. Это нам понадобится позднее.

Визуализация распределения атрибутов и связь с целевой переменной

Для визуализации распределения категориальных атрибутов воспользуемся самым простым инструментом - гистограммой. Построим гистограмму распределения первого существенного атрибута:

```
sns.histplot(data=CS_data, x="channel_name")
```

Обратите внимание, что категории на гистограмме располагаются в порядке упоминания в данных. Когда их много, это может быть неудобно. На гистограмме видна неравномерность распределения - объектов одной категории сильно больше, чем других:



Это уже информацию можно вывести в табличном виде. Для подсчета количества объектов со всеми возможными вариантами значения по определенной колонке в pandas существует специальная функция:

```
CS_data.channel_name.value_counts()
```

Можно просто вывести эту информацию в виде серии. Обратите внимание, что функция сразу и ранжирует категории в порядке убывания частот:

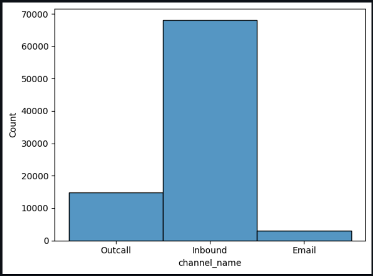
```
channel_name
Inbound    68142
Outcall    14742
Email       3023
Name: count, dtype: int64
```

Эту серию можно использовать, чтобы строить гистограммы, либо тепловые карты. Постройте тепловые карты по всем атрибутам самостоятельно.

Сразу же построим совместное распределение данного признака и целевой переменной. Так как признак категориальный, а целевая переменная - численная, мы можем построить на столбчатой диаграмме среднее значение целевой переменной для каждой категории:

```
sns.catplot(data=CS_data, x="channel_name", y="CSAT Score", kind="bar")
```

На графике видно, что средние значения для первый двух категорий отличаются несущественно, а по третьей - сильно ниже. Можно сделать вывод, что канал обращения влияет на потенциальную удовлетворенность клиента и обращения по электронной почте в среднем немного меньше нравятся клиентам:



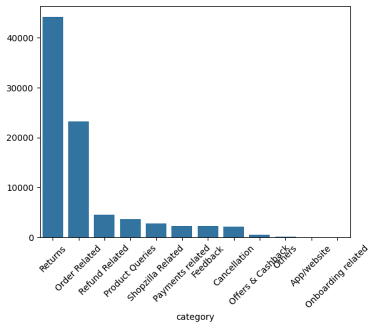
Переходим к следующей категориальной переменной. Прежде чем строить гистограмму выведем таблицу значений:

```
category
Returns                 44097
Order Related           23215
Refund Related           4550
Product Queries          3692
Shopzilla Related        2792
Payments related         2327
Feedback                 2294
Cancellation             2212
Offers & Cashback         480
Others                     99
App/website                84
Onboarding related         65
Name: count, dtype: int64
```

Если категорий очень много, таблицу значений читать проще, так как можно ограничиться анализом только самых распространенных значений. В данном случае значений не так много и можно построить гистограмму. Но мы построим ее по упорядоченному набору значений:

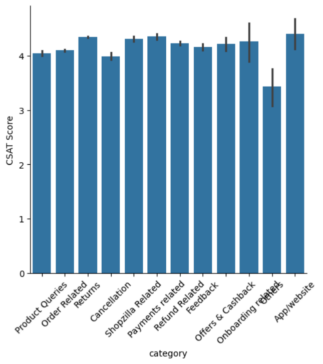
```
counts = CS_data.category.value_counts()
sns.barplot(x=counts.index, y=counts.values)
plt.xticks(rotation=45)
plt.show()
```

Это дает нам более системное представление и соотношении частот разных категорий, когда категории упорядочены по этом частотам:



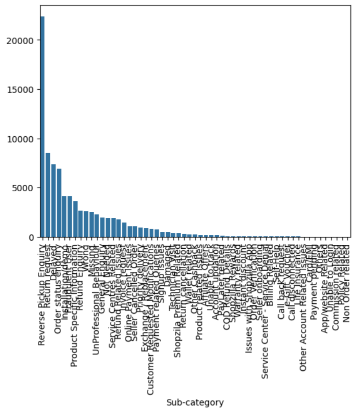
В данном случае, наблюдается типичный для таких распределений дисбаланс категорий. Присутствуют очень малочисленные категории. В таких случая стоит подумать над объединением категорий. Хотя представленный на графике случай еще далеко не самый экстремальный.

Теперь выведем среднее значение удовлетворенности в зависимости от значения данного признака:



Можем видеть наличие влияния: есть разница между средним значением целевой переменной для разных категорий. При этом есть категория, которой соответствует очень низкое значение удовлетворенности. Из этой аналитики тоже можно сделать значимые выводы.

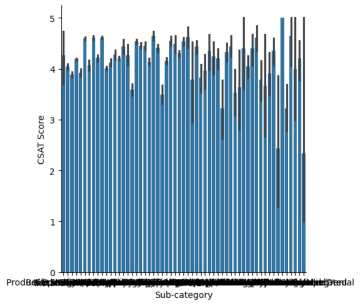
Теперь визуализируем распределение по подкатегориям. Заметно, что при увеличении количества категорий анализ становится проводить затруднительно:



Но в целом, мы видим такое же характерное распределение - очень популярное значение и большое количество редких разных значений. Такие объемные распределения как раз более удобно анализировать в виде таблицы, так как можно вывести первые несколько строк:

```
Sub-category
Reverse Pickup Enquiry          22389
Return request                   8523
Delayed                          7388
Order status enquiry             6922
Installation/demo                4116
Fraudulent User                  4108
Product Specific Information     3589
Refund Enquiry                   2665
Wrong                            2597
Missing                          2556
Name: count, dtype: int64
```

Визуализация совместного распределения с целевой переменной тоже выглядит довольно страшно:



Казалось бы, по этому графику ничего нельзя понять. Но можно сделать вывод: чем менее популярное значение, там больше дисперсия (обозначенная на графике черной вертикальной линией) целевой переменной. Этого и следовало ожидать.

Теперь выведем информацию о столбце "Customer remarks". Так как это текст в свободной форме, и присутствует очень большое количество значений, нет смысла выводить гистограмму. Лучше как раз при помощи таблицы посмотреть несколько самых популярных значений:

```
Customer Remarks
Good             1390
Good             1158
Very good         569
Nice              316
Thanks            276
Ok                259
No                258
Thank you         244
Nice              239
Very good         236
Excellent         171
Thanks            159
Good ??           148
```

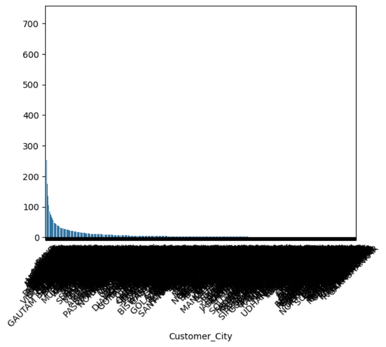
```
Good service      133
Very nice         122
Thank you          97
??                 95
Nothing            88
5                  76
Good job           71
Name: count, dtype: int64
```

Помним, что по этому полю присутствует большое количество пропущенных значений. Можем сделать вывод, что если отзыв присутствует, то с большой долей вероятности он положительный. Поэтому на будущее заметим, что этот атрибут будет логично бинаризовать.

Распределение по городам тоже выведем в виде таблицы:

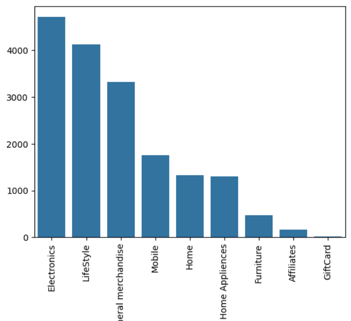
```
Customer_City
HYDERABAD      722
NEW DELHI      688
PUNE           435
MUMBAI         406
BANGALORE      352
CHENNAI        271
KOLKATA        270
LUCKNOW        254
AHMEDABAD      253
JAIPUR         243
GURGAON        215
PATNA          199
SURAT          175
ALLAHABAD      161
KANPUR         138
VARANASI       137
THANE          129
GHAZIABAD      120
BHUBANESWAR    117
VADODARA       105
Name: count, dtype: int64
```

Но чтобы хоть как-то понять общую форму распределения, все-таки изобразим гистограмму:

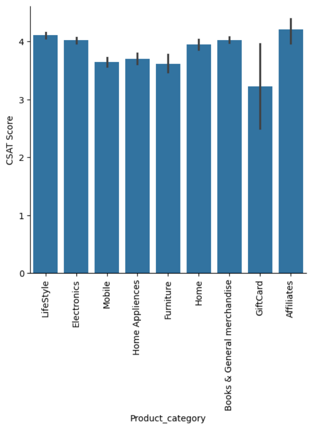


Явно наблюдается экспоненциальное распределение. Несколько очень популярных городов и огромное количество очень редких. Хотя, глядя на таблицу, складывается ощущение, что частоты падают довольно плавно.

Распределение по продуктовой категории не такое экстремальное. Конечно, есть более и менее популярные категории, но в целом дисбаланс не такой выраженный:



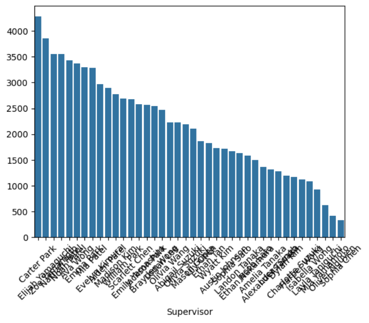
А вотвлияние на целевую переменную здесь не очень очевидно:



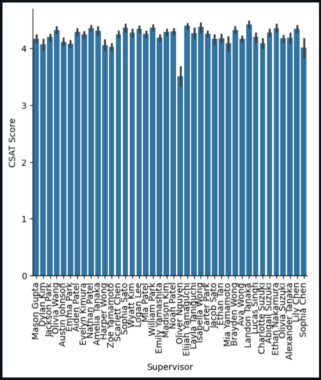
Распределение по операторам опять же выведем только в виде первых строк таблицы. Видим, что значений очень много и распределены они неравномерно:

```
Agent_name
Wendy Taylor          429
Timothy Huff          265
David Smith           264
Jamie Smith           253
Kayla Wilson          216
Julie Williams        200
Mrs. Jennifer Stone   200
Sharon Bullock        195
Matthew White PhD     192
Anthony Booth         177
Tina Harrington       177
Kristin Campbell      176
Brianna Wolf          176
Rebecca Walker        176
Jennifer Hernandez    174
Rebecca Graham        173
William Carey DVM     169
Ryan Thompson         167
Brandon Frost         161
Brian Young           160
Name: count, dtype: int64
```

Руководителей операторов меньше. На гистограмме видно, что дисбаланс присутствует, но не очень яркий:

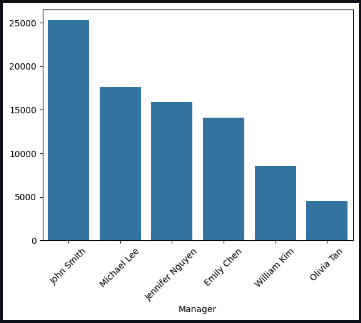


При этом явны различия в средней оценке для разных руководителей:

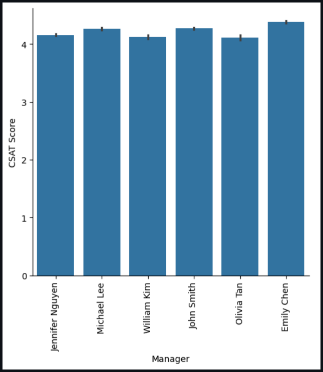


Наверняка есть такие же различия и по операторам, нона графике это увидеть довольно сложно. Попробуйте самостоятельно создать серию, где индексом будет имя оператора, а значением - средняя оценка удовлетворенности. Посчитайте статистику по этой серии.

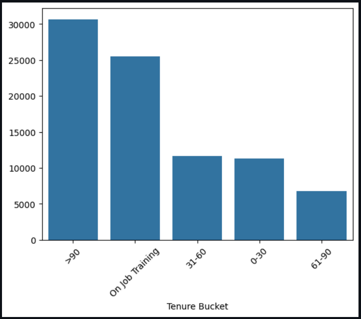
Количество менеджеров еще меньше и картина похожая на руководителей:



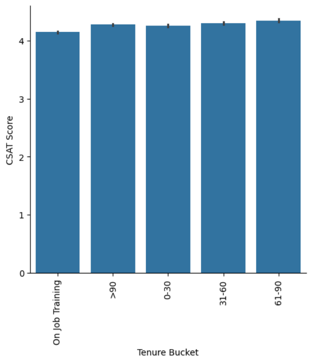
Но различия в целевой переменной у разных менеджеров не такое явное. Видимо, это уже слишком крупная группировка и все различия усредняются:



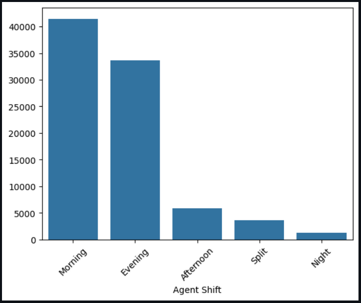
Распределение по типам контракта тоже не вызывает особых вопросов. Преобладают либо долгосрочные контракты, либо сотрудники на обучении:



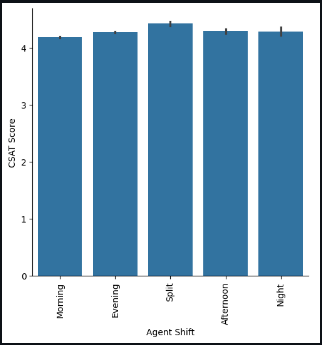
При этом у стажеров средняя оценка совсем незначительно ниже, чем у других категорий, по стальным значимых различий нет:



По типам смен явно преобладают классические утренние и вечерние смены, гибридные варианты достаточно редки:



Есть небольшая вариация в средних значениях удовлетворенности, но ничего особенно примечательного:



В целом, анализ признаков дал нам некоторую информацию о возможных способах преобразования категориальных переменных, чем мы сейчас и займемся. Для более детального анализа влияния атрибутов на целевую переменную в данном случае понадобится дополнительный анализ. Его мы освоим в одной из следующих работ.

Укрупнение категорий

При преобразовании категориальных данных основной проблемой зачастую является то, что при кодировании атрибута с большим количеством значений по методу One-Hot в результате может получиться слишком много столбцов, что вредно как для потенциального переобучения модели, так и не очень эффективно с плане вычислительных ресурсов.

Поэтому при возможности можно постараться укрупнить категории, то есть объединить близкие по смыслу значения атрибута. Посмотрим, например, на отзывы клиента. При анализе мы уже пришли к выводу, что если отзыв присутствует, то он скорее всего положительный. Поэтому можно бинаризировать этот атрибут - если отзыв есть, ставим 1, если нет - 0:

```
CS_data["Customer Remarks New"] = (CS_data["Customer Remarks"].str.len() > 3).astype(int)
```

В данном случае мы проверяем длину строки, чтобы отсечь пустые отзывы, часть их которых обозначена в данных как "NAN". При желании можно совершить более подробную обработку данных. И как всегда советуем создавать новые столбцы:

```
0        0
1        0
2        0
3        0
4        0
        ..
85902    0
85903    1
85904    1
85905    0
85906    0
Name: Customer Remarks New, Length: 85907, dtype: int64
```

Точно также можно бинаризировать и наличие заказа:

```
CS_data["Is_order"] = (CS_data["Order_id"].isna()).astype(int)
```

Дополнительным плюсом такого метода обработки является то, что мы заодно и избавляемся от пропусков в данных:

```
0        0
1        0
2        0
3        0
4        0
        ..
85902    0
85903    0
85904    0
85905    0
85906    0
Name: Is_order, Length: 85907, dtype: int64
```

А вот с другими категориальными переменным так просто уже не поступишь. Например, подкатегория обращения. Здесь мы видим большое количество малопопулярных значений. Часто в такой ситуации можно объединить редкие значения в одно, "Иное". Можно сделать это, например, так:

```
CS_data.loc[~CS_data["Sub-category"].isin([
    "Reverse Pickup Enquiry", "Return request", "Delayed", "Order status enquiry",
    "Installation/demo", "Fraudulent User", "Product Specific Information"
    ]), "Sub-category"] = "Other"
```

Здесь мы явно перечисляем самые популярные категории. Но можно сделать и автоматическое переименование, например, по порогу количества значений. Можете поэкспериментировать с этим самостоятельно. А мы пока заметим, что новое значение стало вторым по популярности в столбце:

```
Sub-category
```

```
Other                          28872
Reverse Pickup Enquiry         22389
Return request                  8523
Delayed                         7388
Order status enquiry            6922
Installation/demo               4116
Fraudulent User                 4108
Product Specific Information    3589
Name: count, dtype: int64
```

Объединение редких значений - отличный способ радикально сократить количество столбцов в датасете после кодирования номинальных признаков. Однако, нужно проверять эффективность данного приема на метриках, и выбирать такое объединение, которое как можно меньше вредит эффективности модели. Как правило, чем больше значений мы объединяем, тем больше потери информации и потенциальное снижение качества модели.

Добавление агрегированной информации

При работе с категориальными переменными с большим количеством значений часто бывает полезно рассмотреть добавление агрегированной информации к данным вместо самой категориальной переменной. Например, можно вместо имени оператора технической поддержки добавить колонку, отражающую загруженность данного оператора. Вполне логично выдвинуть гипотезу о том, что загруженность оператора влияет на качество его работы и, как следствие, на удовлетворенность клиентов.

Можно вывести информацию о количестве записей в колонке, сгруппированных по имени оператора:

```
CS_data.groupby(["Agent_name"]).agg({'Agent_name': 'count'})
```

Это информация нам нужна для проверки предположения о том, что у операторов существенно различается загрузка.

А для того, чтобы добавить колонку с количеством клиентов у соответствующего операторам можно вот так скомбинировать несколько функций pandas:

```
CS_data['Agent_count'] = CS_data.groupby(["Agent_name"])["Agent_name"].transform('count')
```

В итоге мы получаем новый численный признак:

index	Unique id	channel_name	category	Sub-category	Customer Remarks	Order_id	order_date_time	Issue_
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Other	NaN	c27c9bb4-fa36-4140-9f1f-21009254ffdb	NaN	01/08 11:13
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	NaN	d406b0c7-ce17-4654-b9de-f08d421254bd	NaN	01/08 12:52
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/demo	NaN	c273368d-b961-44cb-beaf-62d6fd6c00d5	NaN	01/08 20:16
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	NaN	5aed0059-55a4-4ec6-bb54-97942092020a	NaN	01/08 20:56
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Other	NaN	e8bed5a9-6933-4aff-9dc6-ccefd7dcde59	NaN	01/08 10:30

Точно так же можно поступить и с количеством операторов у одного руководителя. Но теперь нам понадобится другая агрегирующая функция, так как значения имени оператора могут повторяться:

```
CS_data.groupby(["Supervisor"]).agg({'Agent_name': 'nunique'})
```

Вот так выглядит управленческая загрузка руководителей:

Supervisor	Agent_name
Abigail Suzuki	38
Aiden Patel	41
Alexander Tanaka	15
Amelia Tanaka	19
Austin Johnson	29
Ava Wong	70

Brayden Wong	45
Carter Park	64
Charlotte Suzuki	22
Dylan Kim	17
Elijah Yamaguchi	59
Emily Yamashita	42
Emma Park	55
Ethan Nakamura	25
Ethan Tan	31
Evelyn Kimura	39
Harper Wong	22
Isabella Wong	17
Jackson Park	46
Jacob Sato	29
Landon Tanaka	28
Layla Taniguchi	14
Lily Chen	25
Logan Lee	39
Lucas Singh	20
Madison Kim	37
Mason Gupta	41
Mia Patel	62
Mia Yamamoto	8
Nathan Patel	50
Noah Patel	50
Oliver Nguyen	6
Olivia Suzuki	44
Olivia Wang	28
Scarlett Chen	31
Sophia Chen	5
Sophia Sato	22
William Park	36
Wyatt Kim	32
Zoe Yamamoto	68

По аналогии можем добавить эту информацию к исходному датасету:

```
CS_data['Sups_no_agents'] = CS_data.groupby(["Supervisor"])["Agent_name"].transform('nunique')
```

В каждой задаче нужно оценивать необходимость использования агрегированной информации исходя из смысла атрибутов и предметной области. Но в целом это хороший источник новых данных и признаков, не увеличивающий кратно объемов датасетов.

Заполнение отсутствующих значений

В отличие от численных атрибутов, заполнение пропусков в категориальных переменных значительно проще. В большинстве случаев наиболее адекватным будет заполнение неизвестных значений специальным:

```
CS_data['Product_category'] = CS_data['Product_category'].fillna('unknown')
```

Хотя, в некоторых случаях, применяют и заполнение модой, то есть самым распространенным значением. Эффективность конкретных способов нужно оценивать исходя из их влияния на метрики эффективности моделей.

В данном примере еще требуется заполнить пропуски в численном поле. Просто проиллюстрируем, что в особых случаях, численные переменные тоже можно заполнять специальными значениями:

```
CS_data['connected_handling_time'] = CS_data['connected_handling_time'].fillna('0')
```

Конечно это работает только в тех случаях, когда имеет предметный смысл.

Преобразование бинарных атрибутов

Для иллюстрации методов преобразования категориальных переменных в численные нам больше подойдет второй датасет, прилагающийся к данной работе - Титаник. Подробное описание этого уже хорошо известного нам набора мы опустим. Важно, что он содержит категориальные переменные всех интересующих нас типов шкал.

Начнем с самого простого типа категориальных атрибутов - бинарных. Их можно преобразовывать как угодно. Самый компактный способ - LabelEncoder:

```
from sklearn.preprocessing import LabelEncoder
LE_sex = LabelEncoder()
T_data.Sex = LE_sex.fit_transform(T_data.Sex)
```

Этот способ кодирования просто заменяет значения атрибута на последовательные целые числа. В данном случае, 0 или 1 в произвольном порядке:

index	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarkec
0	1	0	3	Braund, Mr. Owen Harris	1	22.0	1	0	A/5 21171	7.25	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	0	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	0	26.0	0	0	STON/O2. 3101282	7.925	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0	35.0	1	0	113803	53.1	C123	S
4	5	0	3	Allen, Mr. William Henry	1	35.0	0	0	373450	8.05	NaN	S

Преобразование порядковых атрибутов

Порядковые атрибуты, то есть такие, значения которых можно сравнивать друг с другом, чаще всего приходится заменять с помощью вручную составленного словаря, чтобы соблюсти естественный порядок значений. Рассмотрим, например, класс обслуживания на Титанике. Его можно закодировать так:

```
T_data.Pclass.replace({
    'first': 1, 'second': 2, 'third': 3
}, inplace=True)
```

После преобразования датасет выглядит следующим образом:

index	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarkec
0	1	0	3	Braund, Mr. Owen Harris	1	22.0	1	0	A/5 21171	7.25	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	0	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	0	26.0	0	0	STON/O2. 3101282	7.925	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0	35.0	1	0	113803	53.1	C123	S
4	5	0	3	Allen, Mr. William Henry	1	35.0	0	0	373450	8.05	NaN	S

Заметим, что такие атрибуты можно кодировать и следующим способом, который является самым универсальным. Но это сопряжено с излишними издержками по памяти.

Преобразование номинальных атрибутов

Номинальные атрибуты, к которым относится большинство категориальных, можно преобразовывать только с использованием OneHotEncoder. Этот способ кодирования следует выбирать в случае сомнений в типе шкалы атрибута. Продемонстрируем работу этого кодировщика:

```
from sklearn.preprocessing import OneHotEncoder
OH_embarked = OneHotEncoder(sparse_output=False)
OH_embarked.fit_transform(T_data[['Embarked']])
```

Как мы видим, интерфейс его работы схож с другимм классами для преобразования данных библиотеки sklearn.

Для формирования таблицы с преобразованными данными нам понадобится сформировать список названий колонок:

```
OH_embarked.get_feature_names_out(['Embarked'])
```

Эту информацию мы передаем конструкторы датафреймов в качестве названий столбцов и так формируем преобразованные данные:

```
dummies = pd.DataFrame(OH_embarked.fit_transform(T_data[['Embarked']]),
                       columns=OH_embarked.get_feature_names_out(['Embarked']),
                       index = T_data.index)
```

Вот так выглядит сгенерированная таблица закодированных значений атрибута:

index	Embarked_C	Embarked_Q	Embarked_S	Embarked_nan
0	0.0	0.0	1.0	0.0
1	1.0	0.0	0.0	0.0
2	0.0	0.0	1.0	0.0
3	0.0	0.0	1.0	0.0
4	0.0	0.0	1.0	0.0

На жаном примере достаточно очевидна схема работы этого кодировщика. Они преобразует одну колонку с множеством значений в множество колонок с бинарными значениями.

В этом виде они готовы к объединению с исходным датафреймом:

```
T_dummies = pd.concat([T_data, dummies]).drop(["Embarked"], axis=1)
```

И в результате мы получаем датафрейм с увеличенным количеством колонок:

index	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarkec
0	1.0	0.0	3.0	Braund, Mr. Owen Harris	1.0	22.0	1.0	0.0	A/5 21171	7.25	NaN	NaN
1	2.0	1.0	1.0	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	0.0	38.0	1.0	0.0	PC 17599	71.2833	C85	NaN
2	3.0	1.0	3.0	Heikkinen, Miss. Laina	0.0	26.0	0.0	0.0	STON/O2. 3101282	7.925	NaN	NaN
3	4.0	1.0	1.0	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0.0	35.0	1.0	0.0	113803	53.1	C123	NaN
4	5.0	0.0	3.0	Allen, Mr. William Henry	1.0	35.0	0.0	0.0	373450	8.05	NaN	NaN

Помните, что использование этого кодировщика может существенно увеличить объем данных.

Удаление лишних столбцов и массовое преобразование

Для финализации процедуры подготовки категориальных данных вернемся к первому датасету. На примере Титаника мы познакомились с подробным алгоритмом работы кодировщика OneHotEncoder. Из-за его универсальности, его часто применяют массово, на всех категориальных атрибутах, независимо от типа. Именно для этого в pandas и существует функция get_dummies().

Но перед ее применением необходимо сначала удалить из датасета все ненужные колонки, особенно категориальные. После преобразования это будет сделать уже не так просто. Мы сейчас удалим все лишние атрибуты произвольно. На практике для этого можно использовать анализ значимости атрибутов.

```
CS_dropped = CS_data.drop([
    "Unique id",
    "Sub-category",
    "Customer Remarks",
    "Customer_City",
    "Agent_name",
    "Supervisor",
    "Order_id",
    "order_date_time",
    "Issue_reported at",
    "issue_responded",
    "Survey_response_Date",
    "Item_price",

], axis=1)
```

Наш датасет принимает более компактную форму:

index	channel_name	category	Product_category	connected_handling_time	Manager	Tenure Bucket	Agent Shift	CSAT Score
0	Outcall	Product Queries	unknown	0	Jennifer Nguyen	On Job Training	Morning	5
1	Outcall	Product Queries	unknown	0	Michael Lee	>90	Morning	5
2	Inbound	Order Related	unknown	0	William Kim	On Job Training	Evening	5
3	Inbound	Returns	unknown	0	John Smith	>90	Evening	5
4	Inbound	Cancellation	unknown	0	Michael Lee	0-30	Morning	5

Перед массовым преобразованием рекомендуем вывести основную статистику по категориальным колонкам еще раз, чтобы убедиться, что в датасете не осталось атрибутов с большим количеством значений:

index	channel_name	category	Product_category	connected_handling_time	Manager	Tenure Bucket	Agent Shift	CSAT S
count	85907	85907	85907	85907	85907	85907	85907	85907.0
unique	3	12	10	212	6	5	5	NaN
top	Inbound	Returns	unknown	0	John Smith	>90	Morning	NaN
freq	68142	44097	68711	85665	25261	30660	41426	NaN
mean	NaN	NaN	NaN	NaN	NaN	NaN	NaN	4.24215721
std	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1.37890305
min	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1.0
25%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	4.0
50%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	5.0
75%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	5.0
max	NaN	NaN	NaN	NaN	NaN	NaN	NaN	5.0

Зодно проверим, что заполнены либо удалены все пропуски в данных.

```
RangeIndex: 85907 entries, 0 to 85906
Data columns (total 12 columns):
 #   Column                   Non-Null Count  Dtype 
---  ------                   --------------  ----- 
 0   channel_name             85907 non-null  object
 1   category                 85907 non-null  object
 2   Product_category         85907 non-null  object
 3   connected_handling_time  85907 non-null  object
 4   Manager                  85907 non-null  object
 5   Tenure Bucket            85907 non-null  object
 6   Agent Shift              85907 non-null  object
 7   CSAT Score               85907 non-null  int64 
 8   Customer Remarks New     85907 non-null  int64 
 9   Is_order                 85907 non-null  int64 
 10  Agent_count              85907 non-null  int64 
 11  Sups_no_agents           85907 non-null  int64 
dtypes: int64(5), object(7)
memory usage: 7.9+ MB
```

Теперь одной инструкцией можно массово преобразовать все категориальные атрибуты в бинарные признаки по методу OneHotEncoder:

```
CS_dummies = pd.get_dummies(CS_dropped)
```

После этого преобразования количество столбцов в датасете может произвольно увеличиться. В данном случае мы проконтролировали этот процесс, но все равно получили в несколько раз больше колонок, чем было:

```
(85907, 258)
```

В реальности увеличение объема данных может быть еще более значительным. Если это нежелательно, нужно работать с категориальными переменными до преобразования, как мы показали в этой работе.

Теперь после такого преобразования данные полностью готовы к тому, чтобы их использовать в любой модели машинного обучения.

Задания для самостоятельного выполнения

1. Постройте визуализацию распределения признаков и совместного распределения признаков и целевой переменной с помощью тепловых карт. Попробуйте использовать другие инструменты визуализации.
2. Постройте на получившимся датасете Customer support модель дерева решений и проанализируйте важность признаков. Сделайте вывод об адекватности наших предположений.
3. Разбейте датасет на тестовую и обучающую выборки и преобразуйте обе подвыборки. Тестовую нужно преобразовывать точно также, как и обучающую (с теми же параметрами).
4. Проведите полный анализ на датасете Титаник, включая все необходимые визуализации и выводы.

5. Проверьте целесообразность каждого необязательного преобразования данных путем проверки, увеличивает ли данное преобразование точность модели. Проверьте на простом виде модели (линейная регрессия, дерево решений или случайный лес). Поэкспериментируйте с различными вариантами преобразований.
6. Создайте воспроизводимый код обработки данного датасета.

Контрольные вопросы

1. Какие основные типы графиков используются для визуализации эмпирического распределения категориальных атрибутов?
2. В чем разница между столбчатой диаграммой и гистограммой?
3. Какие графики целесообразно использовать для визуализации совместного распределения категориального атрибута и целевой переменной?
4. Какие способы заполнения отсутствующих значений работают с категориальными признаками?
5. Какие виды категориальных признаков существуют? Чем они определяются?
6. Как следует преобразовывать категориальные признаки в численные?

Дополнительные задания

1. Проведите полный анализ и преобразование категориальных атрибутов на датасете, который использовался в предыдущей работе.
2. (*) Создайте код, реализующий алгоритм очистки данных автоматически (для любого датасета).